RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering (Cyber Security)

CS23533– Foundations of Artificial Intelligence

2026-2027

MITRE ATTACK RULE-BASED NAVIGATOR REVIEW 2

Supervisor:
Dr. S. Bhuvana
Assistant Professor [SG]

Batch Members:
Pranathee H – 241901078
Sivaranjanii M – 241901109

Problem Statement and Motivation

Problem

- Analysts see raw system behaviours (process creation, registry writes, failed logins)
- They must map these to MITRE ATTACK techniques by hand, which is slow and inconsistent

Motivation

- MITRE ATTACK is the public standard used by security teams worldwide
- Alerts only help when tied to a technique and a response action
- Manual mapping does not scale as the volume of events grows

Objectives

- Build a **rule-based expert system** that maps observed behaviours (PowerShell execution, registry modification, repeated failed logins) to likely **ATTACK techniques**
- Output the detected technique, its risk level, and recommended mitigation steps
- Use a **Goal-Based Intelligent Agent** to control reasoning, with IF-THEN rules representing ATT&CK knowledge
- Apply **Best-First Search** to find relevant rules and **Forward Chaining** to infer the technique
- Implement in **Python** using classical AI only, with no Machine Learning or Deep Learning

Project Progress Since Review 1

- Built the rule base (IF-THEN rules in rules.json) and mitigation data (mitigations.json)
- Implemented Best-First Search and Forward Chaining in Python
- Built the goal-based agent that ties the modules together, with sample inputs and test scenarios
- Output shows technique, risk level, mitigation and an explanation trace
- Added a CLI and a Flask web interface

Literature Survey

S. No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
1	B. E. Strom et al. (2018)	MITRE ATT&CK: Design and Philosophy	Knowledge base of adversary tactics and techniques (TTPs) built from observed real-world behaviour	A reference catalogue only; no runtime reasoning that maps live observations to techniques
2	U. Lindqvist & P. A. Porras (1999)	Detecting Computer and Network Misuse Through the Production-Based Expert System Toolset (P-BEST)	Forward-chaining production rules for misuse detection	Predates ATT&CK; rules not tied to a standard framework, no mitigation guidance, no rule prioritisation
3	G. Husari et al. (2017)	TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources	Ontology and NLP-based mapping of threat-report text to ATT&CK techniques	Works on written reports after an incident, not on live observed behaviour

Literature Survey

S. No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
4	P. E. Hart, N. J. Nilsson & B. Raphael (1968)	A Formal Basis for the Heuristic Determination of Minimum Cost Paths	Heuristic best-first search with a priority queue (A*)	General-purpose; needs a domain heuristic to rank security hypotheses
5	C. L. Forgy (1982)	Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem	Efficient pattern matching for production (rule-based) systems	Speeds up matching only; does not rank partial matches or supply security knowledge

Research Gap and Project Response

Existing work	What it gives	What is missing
ATT&CK [1]	Standard vocabulary of techniques	Only a catalogue; no reasoning on live events
P-BEST [2]	Forward-chaining rules for misuse detection	No ATTACK link, no mitigations, no ranking
TTPDrill [3]	Text reports mapped to ATTACK	Works after the incident, on text only
A* [4], Rete [5]	Heuristic ranking and fast rule matching	No security knowledge; no ranking of partial matches

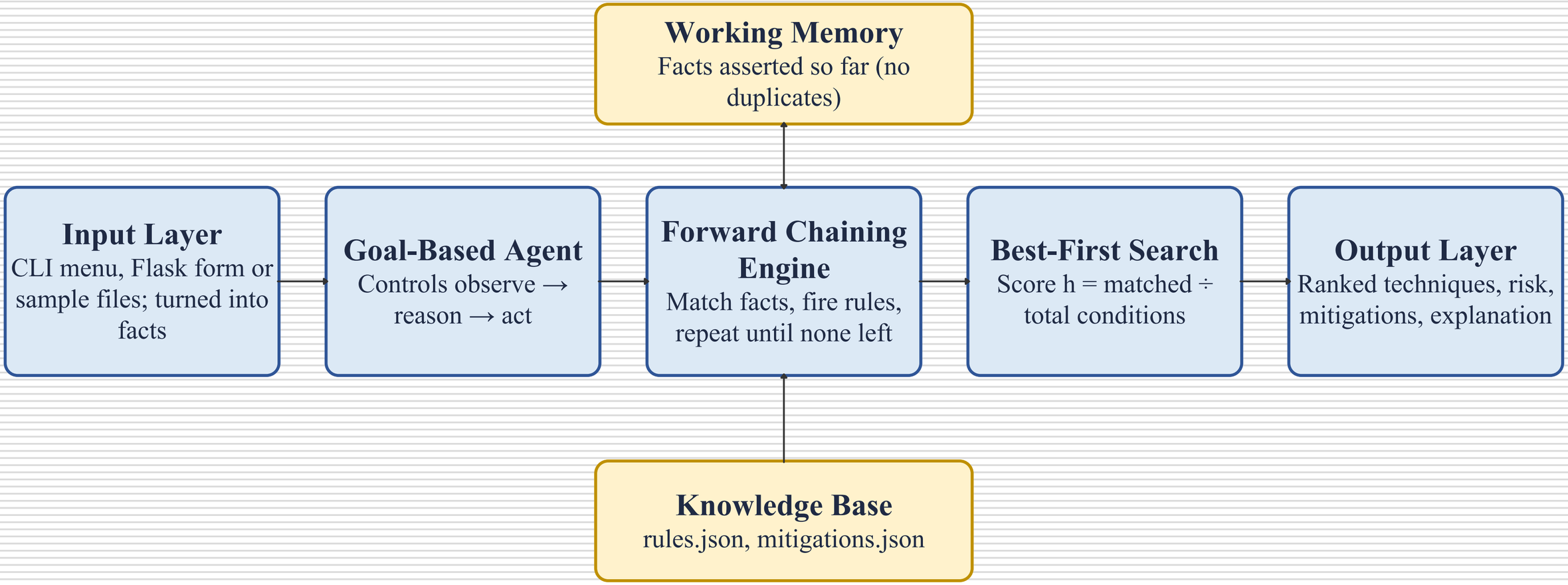
Refined research gap

No lightweight, explainable system maps observed behaviour directly to ATTACK, ranks the possible techniques by how strong the evidence is, and recommends mitigations.

Progress toward the solution

- IF–THEN rule base and forward-chaining engine
- Best-first ranking: score = matched ÷ total conditions; partial matches shown as "possible"
- Risk level, MITRE mitigations and explanation trace
- CLI and Flask interfaces; Python only, offline

System Architecture / Proposed System



Improvements over existing systems: heuristic prioritisation instead of sequential rule scanning, explainable decisions instead of black-box ML, built-in mitigations, offline and Python-only.

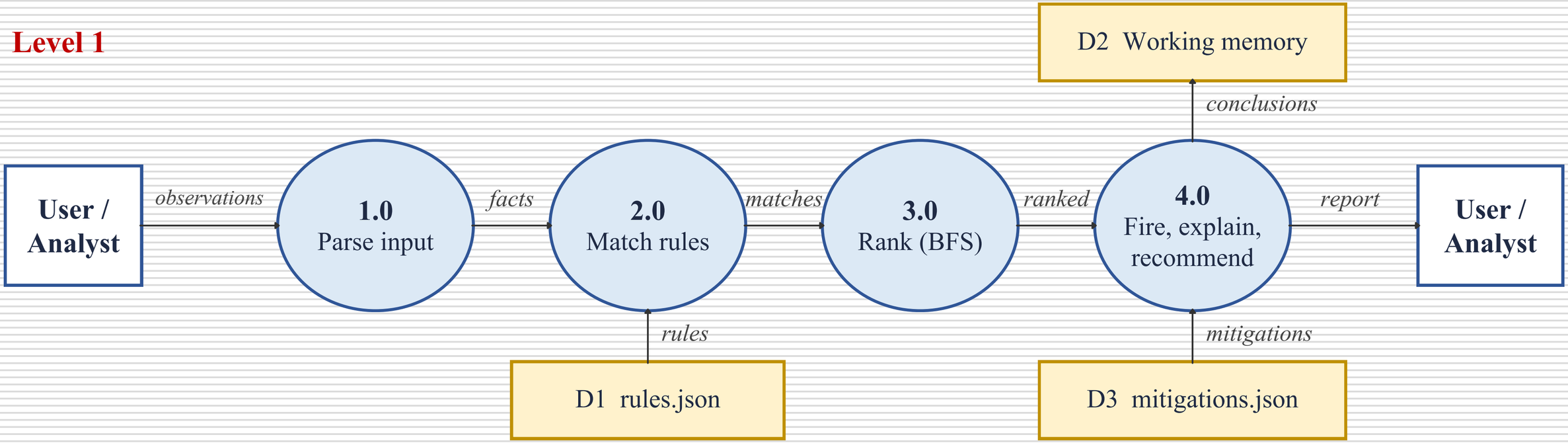
8

DFD Diagram

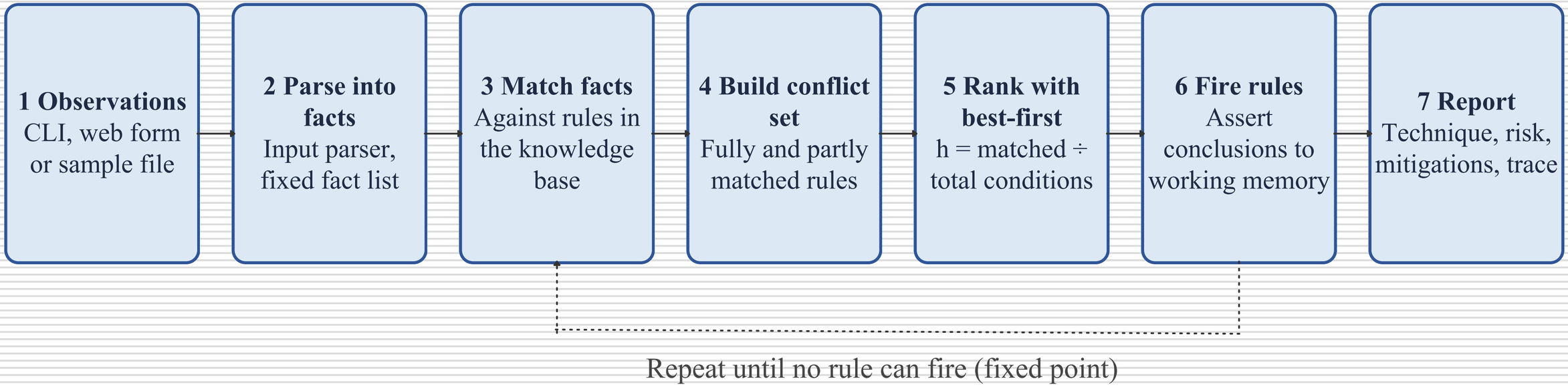
Level 0



Level 1



Methodology / Workflow Diagram



Tools and Technologies

- **Programming language:** Python 3.10+
- **Frameworks / libraries:** heapq, json, dataclasses, collections (standard library); Flask for the web interface; tabulate and colorama for console reports; matplotlib and pandas for optional charts and CSV logs
- **Software:** VS Code, Git and GitHub, virtual environment (venv), Jupyter for demos
- **Knowledge source:** MITRE ATT&CK Enterprise matrix (technique IDs, descriptions, M-code mitigations); no dataset or training data
- **Hardware:** Intel Core i3 or higher, 4 GB RAM (8 GB recommended), no GPU
- **Justification:** no ML libraries because the design premise is explainable classical AI; the standard library keeps it lightweight and offline.

List of Modules

Module	Description
1 Knowledge Base & Facts	Rule schema, KnowledgeBase loader and validation, rules.json, mitigations.json
2 Forward Chaining Engine	Working memory, match phase, conflict set, act phase, explanation trace
3 Best-First Search	Heuristic h(rule), priority queue, tie-breaking, near-miss reporting
4 Agent, Interface & Testing	Agent, input parser, CLI menu, report format, test scenarios, demo
5 Flask Web Interface	Web front end over the same agent and engine
6 Final Testing & Documentation	Accuracy measurement, report, GitHub update

Module Implementation – Part 1

Module 1 – Knowledge Base and Facts

- Fact: one observed behaviour, e.g. failed_logins > 5
- Rule: ID, list of conditions, technique, tactic and risk level
- KnowledgeBase class loads rules.json; rejects broken rules and duplicate IDs
- mitigations.json links each technique to its ATTACK mitigations

Module 2 – Forward Chaining Engine

- Working memory stores facts without duplicates
- Cycle: match rules, collect eligible rules, fire one, add its conclusion
- Counts satisfied conditions per rule (partial match) for Module 3
- Explanation trace: which rule fired, on which facts, in what order

Rule format

```
{
  "id": "R3",
  "if": ["failed_logins > 5"],
  "then": "T1110 Brute Force",
  "tactic": "Credential Access",
  "risk": "High"
}
```

Input: observed behaviours as facts. Output: fired rules, derived techniques and the explanation trace.

Module Implementation – Part 2

Module 3 – Best-First Search

- Score h(rule) = matched conditions ÷ total conditions (0.0 to 1.0)
- Priority queue (heapq) returns the best-scored rule first
- Ties go to the rule with the higher-severity tactic
- Partly matched rules are reported as "possible" techniques

Module 4 – Agent, Interface and Testing

- Agent class: observe, reason (Modules 2 and 3), act by recommending mitigations
- Input from CLI menu and sample files; Flask web interface on the same agent
- Report shows techniques, confidence, evidence and mitigations

Ranking logic (pseudocode)

```
for rule in rules:
  h = matched(rule) / total(rule)
  push(queue, (-h, -severity, rule))

while queue not empty:
  rule = pop(queue)     # best first
  if h == 1.0:
    fire(rule)          # forward chain
    assert conclusion
  elif h > 0:
    report "possible" technique

return techniques ranked by h
```

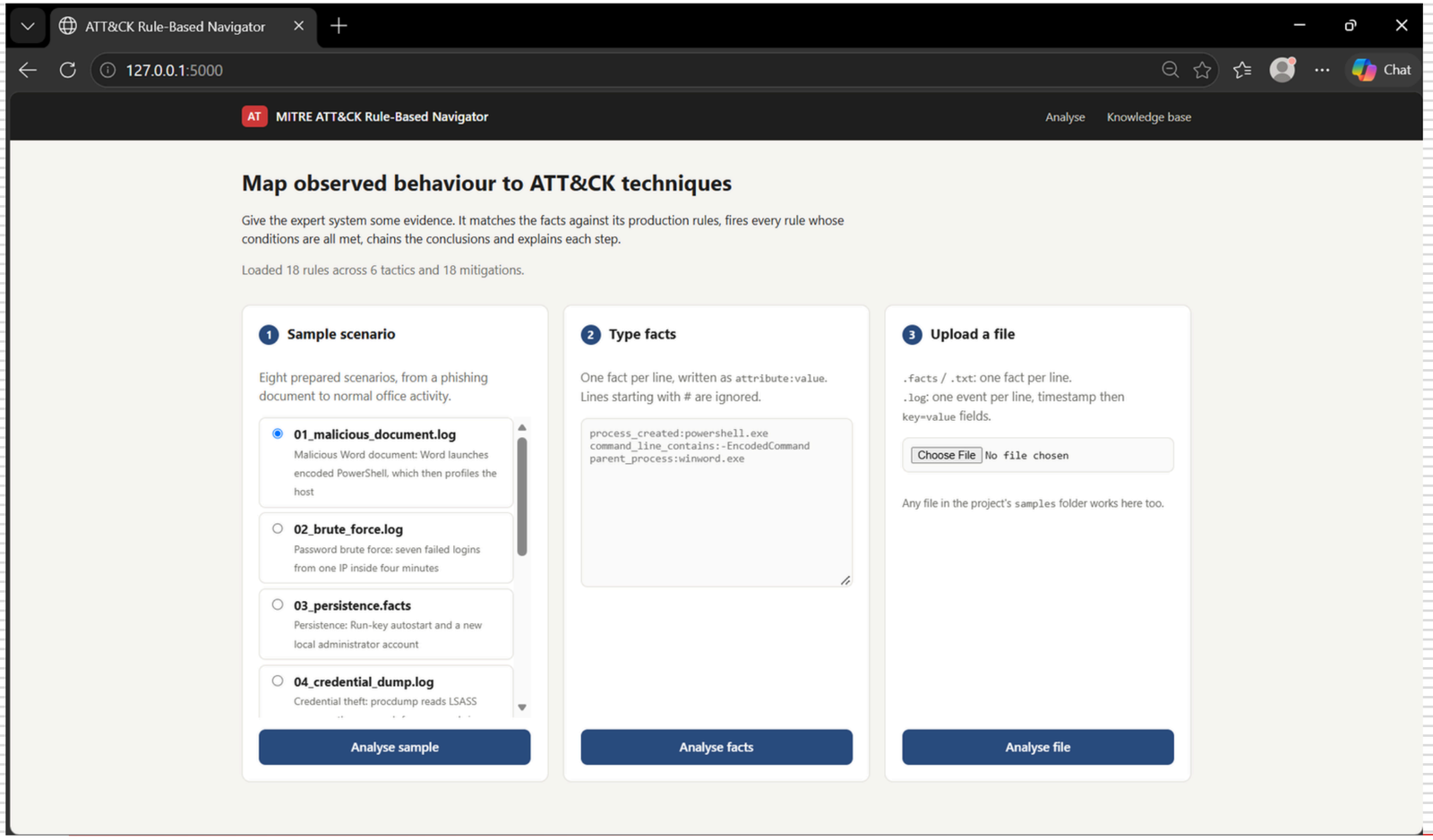
Preliminary Results and Discussion

Scenario	Observed behaviour (facts)	Expected technique	Score	Outcome
PowerShell-based intrusion	powershell.exe spawned, encoded command	T1059.001 PowerShell	**1.00**	Fires (h = 1.0)
Persistence via registry	Run key modified, new startup entry	T1547 Registry Run Keys	**0.67**	Possible (near-miss)
Credential brute force	failed_logins > 5 from one source	T1110 Brute Force	**0.50**	Possible (near-miss)

Discussion

- Results are ranked by confidence, so the best-supported technique comes first.
- Partly matched rules appear as "possible" techniques instead of being dropped.
- Compared with sequential rule scanning, the score puts the most relevant rule first and every result has a reasoning trace.
- Duplicate-free facts and the fixed-point stop prevent rules from firing again and again.

User Interface / Implementation Screenshots



HOME PAGE

AND

INPUT INTERFACE

Detection report

01_malicious_document.log

4 lines read · 8 facts · 2026-10-18 00:24

New analysis

Techniques detected	Highest severity	Near misses	Inference
3	HIGH	**0**	**3 cycles**
high 2 · medium 0 · low 1		partly matched rules	8 observed + 6 derived facts, stopped by quiescence

Tactics reached

Initial Access → Execution → Discovery

Detected techniques ranked by confidence, then severity

#	TECHNIQUE	NAME	TACTIC	SEVERITY	CONFIDENCE	RULE	CYCLE
1	T1059.001	Command and Scripting Interpreter: PowerShell	Execution	HIGH	1.00	R001	1
2	T1566.001	Phishing: Spearphishing Attachment	Initial Access	HIGH	1.00	R018	2
3	T1082	System Information Discovery	Discovery	LOW	1.00	R014	3

ALERT AND DETECTION

Why reasoning behind each detection

```
T1059.001 - Command and Scripting Interpreter: PowerShell
  concluded by rule R001 (cycle 1, tactic: Execution, severity: high)
  because ALL of these facts were in working memory:
    [observed] process_created:powershell.exe
    [observed] command_line_contains:-EncodedCommand
    [observed] parent_process:winword.exe

T1566.001 - Phishing: Spearphishing Attachment
  concluded by rule R018 (cycle 2, tactic: Initial Access, severity: high)
  because ALL of these facts were in working memory:
    [derived by R001] technique_detected:T1059.001
        T1059.001 - Command and Scripting Interpreter: PowerShell
          concluded by rule R001 (cycle 1, tactic: Execution, severity: high)
          because ALL of these facts were in working memory:
            [observed] process_created:powershell.exe
            [observed] command_line_contains:-EncodedCommand
            [observed] parent_process:winword.exe
    [observed] parent_process:winword.exe

T1082 - System Information Discovery
  concluded by rule R014 (cycle 3, tactic: Discovery, severity: low)
  because ALL of these facts were in working memory:
    [observed] process_created:systeminfo.exe
    [observed] command_output_redirected:true
```

MITIGATION

Recommended mitigations

ID	MITIGATION	ADDRESSES
M1042	Disable or Remove Feature or Program	T1059.001
M1026	Privileged Account Management	T1059.001
M1038	Execution Prevention	T1059.001
M1049	Antivirus/Antimalware	T1566.001
M1017	User Training	T1566.001
M9001 *	Detection-Only Guidance (team-defined)	T1082

* M9001 is team-defined, not an official ATT&CK mitigation. ATT&CK lists no preventive mitigations for Discovery techniques.

Challenges Faced and Solutions

- **Vague observations:** real events do not match rule text exactly. Solved with a fixed fact vocabulary and a parser for the input menu.
- **Partial matches:** a flat match/no-match hid useful evidence. Solved with the matched ÷ total heuristic and "possible" reporting.
- **Ties and repeated firing:** equal scores and rules re-firing. Solved with severity tie-breaks, duplicate-free working memory and fixed-point termination.
- **Parallel development:** two people on four modules. Solved with a shared GitHub repo and one module per milestone.
- **Remaining issues:** rule base covers a subset of ATTACK; inputs are Yes/No facts, not raw logs.

Work Plan Until Final Review

- **Phase 1:** push the latest Flask and CLI updates to GitHub and clean the repository.
- **Phase 2:** extend the rule base and add more attack scenarios.
- **Phase 3:** final testing and accuracy measurement against expected techniques; fix issues found.
- **Phase 4:** final report and documentation.

References

- [1] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, 2018 (revised 2020).
- [2] U. Lindqvist and P. A. Porras, “Detecting Computer and Network Misuse Through the Production-Based Expert System Toolset (P-BEST),” Proc. IEEE Symposium on Security and Privacy, 1999.
- [3] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu and X. Niu, “TTPDrill: Automatic and Accurate Extraction of Threat Actions from Unstructured Text of CTI Sources,” Proc. ACSAC, 2017.
- [4] P. E. Hart, N. J. Nilsson and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, 1968.
- [5] C. L. Forgy, “Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem,” Artificial Intelligence, vol. 19, no. 1, 1982.

Thank You